第1章 状态估计问题

对于运动方程、观测方程：

位姿由变换矩阵描述，然后使用李代数进行优化，观测方程由相机成像模型给出，其中内参随机固定，而外参是相机的位姿。

但是受到噪声的影响，两个方程只能近似成立，因此需要在有噪声的数据中进行准确的状态（位姿、地图）估计。

1.1 状态估计问题

对于运动方程、观测方程，假设假设两个噪声均服从高斯分布：

$$\begin{cases} \boldsymbol{x}_k = f(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k) + \boldsymbol{\omega}_k \\ \boldsymbol{z}_{k,j} = h(\boldsymbol{y}_j, \boldsymbol{x}_k) + \boldsymbol{v}_{k,j} \end{cases}, \quad k = 1, ...N; j = 1, ..., M$$

$$\boldsymbol{\omega}_k \sim N(0, \boldsymbol{R}_k), \quad \boldsymbol{v}_k \sim N(0, \boldsymbol{Q}_{k,j})$$

利用带噪声的数据$\boldsymbol{z}$和$\boldsymbol{u}$推断位姿$\boldsymbol{x}$和地图$\boldsymbol{y}$（以及它们的概率分布）。利用概率表示为求条件概率分布（将待估计状态记作一个状态变量）：

$$\begin{cases} P(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{u}) \\ \boldsymbol{x} = \boldsymbol{x}_1, ..., \boldsymbol{x}_N, \boldsymbol{y}_1, ..., \boldsymbol{y}_M \end{cases}$$

如果没有测量运动的传感器，只有图像，则条件概率分布简化为$P(\boldsymbol{x}|\boldsymbol{z})$，根据贝叶斯法则，有：

$$P(\boldsymbol{x}|\boldsymbol{z}) = \frac{P(\boldsymbol{z}|\boldsymbol{x})P(\boldsymbol{x})}{p(\boldsymbol{z})} \propto P(\boldsymbol{z}|\boldsymbol{x})P(\boldsymbol{x})$$

- 后验概率：$P(\boldsymbol{x}|\boldsymbol{z})$
- 似然：$P(\boldsymbol{z}|\boldsymbol{x})$

- 先验：$P(\boldsymbol{x})$

1.1.1 最大后验

直接求后验概率分布较为困难。

方法：求一个最优估计，使得在该状态下，**后验概率最大化（Maximize a Posterior，MAP）**：

$$\boldsymbol{x}_{*MAP} = argmax(P(\boldsymbol{x}|\boldsymbol{z})) = argmax(P(\boldsymbol{z}|\boldsymbol{x})P(\boldsymbol{x}))$$

1.1.2 最大似然

如果不知道机器人大概在什么位置，即没有先验，则问题为求**最大似然估计（Maximize Likelihood Estimation，MLE）**：

$$\boldsymbol{x}_{\underset{MLE}{*}} = argmax(P(\boldsymbol{z}|\boldsymbol{x}))$$

似然：在现在位姿下，可能产生怎样的观测数据。

最大似然：在什么样的状态下，最可能产生现在的观测数据（观测数据已知）。

1.2 状态估计的概率解释

后端优化中，通常考虑一段时间内（或所有时间内）的状态估计问题，而且不仅使用过去的信息更新自己的状态，也会用未来的信息来更新自己

1.2.1 实际中的运动/观测方程

观测方程多于运动方程：

实际中， 机器人在一个位置通常只能看到一小部分路标。而且，由于视觉或激光 SLAM特征点数量都多，所以实际当中观测方程数量会远远大于运动方程的数量。

如果没有运动方程：

如果没有测量运动的装置， 就没有明确的运动方程。 此时可认为没有运动方程，或假设机器人不动，或机器人匀速运动。

此时，后端优化问题由多个观测方程组成。类似于SfM（Structure from Motion）问题，即通过一组图像来恢复运动和结构。与 SfM 中不同的是，SLAM 中的图像有时间上的先后顺序，而 SfM 中允许使用完全无关的图像。

1.2.2 概率解释

令$\boldsymbol{x}_k$代表k时刻所有未知量，包含当前时刻的位姿、m个路标点。且$\boldsymbol{z}_k$表示k时刻所有的观测。

$$\boldsymbol{x}_k \triangleq \boldsymbol{x}_k, \boldsymbol{y}_1, ..., \boldsymbol{y}_m$$

两个方程改写为如下：

$$\begin{cases} \\ \boldsymbol{x}_k = f(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k) + \boldsymbol{\omega}_k \\ \boldsymbol{z}_k = h(\boldsymbol{x}_k) + \boldsymbol{v}_k \end{cases}, \quad k = 1, ..., N$$

利用过去$0-k$中的数据来估计当期时刻的状态分布：

$$P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k})$$

根据贝叶斯法则，有：

$$P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k}) \propto P(\boldsymbol{z}_k | \boldsymbol{x}_k) P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1})$$

右式第一项为似然，由观测方程给定。第二项为先验，当前状态由过去所有状态估计得到，可将先验依次向前一时刻展开。

1.2.3 两类方法

滤波方法（EKF 等）：

马尔可夫性：k时刻状态只与$k-1$时刻状态有关。将先验按照$k-1$时刻展开如下：

$$P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) = \int P(\boldsymbol{x}_k | \boldsymbol{x}_{k-1}, \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) P(\boldsymbol{x}_{k-1} | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) d\boldsymbol{x}_{k-1}$$

非线性优化方法：

k时刻状态与之前所有状态有关。可将先验继续往前展开。

第2章 非线性优化

2.1 最小二乘的引出

高维高斯分布

对于一个任意的高维高斯分布$\boldsymbol{x}\sim N\left(\boldsymbol{\mu},\boldsymbol{\Sigma}\right)$，它的概率密度函数展开形式：

$$P(\boldsymbol{x})=\frac{1}{\sqrt{\left(2\pi\right)_{N}det(\boldsymbol{\Sigma})}}exp\left(-\frac{1}{2}\left(\boldsymbol{x}-\boldsymbol{\mu}\right)_{T}\boldsymbol{\Sigma}_{-1}\left(\boldsymbol{x}-\boldsymbol{\mu}\right)\right)$$

取其负对数：

$$-\ln\left(P\left(\boldsymbol{x}\right)\right)=\frac{1}{2}\ln\left(\left(2\pi\right)_{N}det(\boldsymbol{\Sigma})\right)+\frac{1}{2}\left(\boldsymbol{x}-\boldsymbol{\mu}\right)_{T}\boldsymbol{\Sigma}_{-1}\left(\boldsymbol{x}-\boldsymbol{\mu}\right)$$

只要最小化上式右侧第二项，就得到原分布最大化，即得到对状态的最大似然估计。

构建最小二乘问题

状态估计问题中，假设运动、观测噪声服从高斯分布，根据运动、观测方程，二者数据也服从高斯分布，且条件概率分别为：

$$\begin{cases}P(\boldsymbol{x}_{k}|\boldsymbol{x}_{k-1},\boldsymbol{u}_{k})=N\left(f(\boldsymbol{x}_{k-1},\boldsymbol{u}_{k}),\boldsymbol{R}_{k}\right)\\P(\boldsymbol{z}_{j,k}|\boldsymbol{x}_{k},\boldsymbol{y}_{j})=N\left(h(\boldsymbol{x}_{k},\boldsymbol{y}_{j}),\boldsymbol{Q}_{k,j}\right)\end{cases}$$

综合考虑二者，定义数据与估计的误差，且有误差的平方和。使该平方和最小的最优解，即为状态的最大似然估计。

$$
\begin{aligned}
f(\boldsymbol{x}) &= \sum e_{v,kT}\boldsymbol{R}_{k-1}e_{v,k} + \sum_k\sum_j e_{y,k,jT}\boldsymbol{Q}_{k,j-1}e_{y,k,j} \\
e_{v,k} &= \boldsymbol{x}_k^k - f(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k) \\
e_{y,k,j} &= \boldsymbol{z}_{j,k} - h(\boldsymbol{x}_k, \boldsymbol{y}_j)
\end{aligned}
$$

2.2 非线性最小二乘

对于如下最小二乘问题，使用迭代方法求解：从一个初始值出发，不断寻找梯度并下降，直到增量非常小，无法再使函数下降，此时算法收敛，目标达到了一个极小。其中对于增量的确定，分为多种方法，如牛顿法、高斯牛顿法、列文伯格-马夸尔特。

$$
\min_x \frac{1}{2}||f(\boldsymbol{x})||_{22}
$$

- 给定某个初始值$\boldsymbol{x}_0$
- 对于第k次迭代，寻找一个增量$\Delta\boldsymbol{x}_k$，使得$||f(\boldsymbol{x}) + \Delta\boldsymbol{x}_k||_{22}$达到极小值
- 若$\Delta\boldsymbol{x}_k$足够小，则停止
- 否则，令$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta\boldsymbol{x}_k$，返回2

2.2.1 一阶和二阶梯度法

推导：

将目标函数在$\boldsymbol{x}$附近进行泰勒展开。其中$\boldsymbol{J}(\boldsymbol{x})$是$||f(\boldsymbol{x})||_{22}$关于$\boldsymbol{x}$的导数（Jacobian矩阵），而$\boldsymbol{H}$是二阶导数（Hessian矩阵）。

$$
||f(\boldsymbol{x}) + \Delta\boldsymbol{x}||_{22} = ||f(\boldsymbol{x})||_{22} + \boldsymbol{J}(\boldsymbol{x})\Delta\boldsymbol{x} + \frac{1}{2}\Delta\boldsymbol{x}_T\boldsymbol{H}\Delta\boldsymbol{x} + \cdots
$$

- 一阶（最速下降法）：取前两项，关于$\Delta\boldsymbol{x}$求导并令其值等于0，可得到增量的解为$\boldsymbol{J}_T(\boldsymbol{x})$，即沿着反向梯度方向前进即可。实际中可加上步长系数，即最快的下降速度（最速下降法）。

$$\Delta \boldsymbol{x}_{*}=-\boldsymbol{J}_{T}(\boldsymbol{x})$$

- 二阶（牛顿法）：取前三项，关于$\Delta \boldsymbol{x}$求导并令为0，得到增量的解如下。

$$\boldsymbol{H}\Delta \boldsymbol{x}=-\boldsymbol{J}_{T}(\boldsymbol{x})$$

优缺点：

- 直观，只需把函数在迭代点附近进行泰勒展开，并针对更新量做小化即可。
- 最速下降法：过于贪心，容易走出锯齿路线，反而增加迭代次数。
- 牛顿法：问题规模很大时，计算目标函数的$\boldsymbol{H}$矩阵非常困难，因此通常需避免$\boldsymbol{H}$的计算，该方法不常用。

2.2.2 高斯牛顿法(G-N)

推导：

- 将$f(\boldsymbol{x})$进行一阶的泰勒展开

$$f(\boldsymbol{x}+\Delta \boldsymbol{x})\approx f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})\Delta \boldsymbol{x}$$

- 将上述展开式代入原始最小二乘问题，构建新的线性最小二乘问题：

$$\Delta \boldsymbol{x}_{*}=arg\min_{\Delta \boldsymbol{x}}\frac{1}{2}\left|\left|f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})\Delta \boldsymbol{x}\right|\right|_{2}$$

- 展开目标函数的平方项：

$$\begin{aligned}\frac{1}{2}\left|\left|f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})\Delta \boldsymbol{x}\right|\right|_{2}&=\frac{1}{2}\left(f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})\Delta \boldsymbol{x}\right)_{T}\left(f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})\Delta \boldsymbol{x}\right)\\&=\frac{1}{2}\left(\left|\left|f(\boldsymbol{x})\right|\right|_{22}+2f(\boldsymbol{x})_{T}\boldsymbol{J}(\boldsymbol{x})\Delta \boldsymbol{x}+\Delta \boldsymbol{x}_{T}\boldsymbol{J}(\boldsymbol{x})_{T}\boldsymbol{J}(\boldsymbol{x})\Delta \boldsymbol{x}\right)\end{aligned}$$

- 求上式关于$\Delta \boldsymbol{x}$的导数，并令其为零，可得到一个线性方程组:

$$2\boldsymbol{J}(\boldsymbol{x})_{T}f(\boldsymbol{x})+2\boldsymbol{J}(\boldsymbol{x})_{T}\boldsymbol{J}(\boldsymbol{x})\Delta \boldsymbol{x}=\boldsymbol{0}$$

$$\boldsymbol{J}(\boldsymbol{x})_T\boldsymbol{J}(\boldsymbol{x})\Delta\boldsymbol{x}=-\boldsymbol{J}(\boldsymbol{x})_Tf(\boldsymbol{x})$$

- 求解正规方程(高斯牛顿方程/增量方程)：将上式左边系数定义为$\boldsymbol{H}$，右边定义为$\boldsymbol{g}$。即高斯牛顿法使用作为$\boldsymbol{H}$（二阶Hessian矩阵）的近似，从而省略$\boldsymbol{H}$的过程。

$$\boldsymbol{H}\Delta\boldsymbol{x}=\boldsymbol{g}$$

总结：

- 给定初始值$\boldsymbol{x}_0$
- 对于第k次迭代，求出当前的雅可比矩阵矩阵$\boldsymbol{J}(\boldsymbol{x}_k)$和误差$f(\boldsymbol{x}_k)$
- 求解增量方程

$$\begin{cases}\boldsymbol{H}\Delta\boldsymbol{x}_k=\boldsymbol{g}\\\boldsymbol{H}=\boldsymbol{J}(\boldsymbol{x}_k)_T\boldsymbol{J}(\boldsymbol{x}_k)\\\boldsymbol{g}=-\boldsymbol{J}(\boldsymbol{x}_k)_Tf(\boldsymbol{x}_k)\end{cases}$$

- 若$\Delta\boldsymbol{x}_k$足够小，则停止。否则，令$\boldsymbol{x}_{k+1}=\boldsymbol{x}_k+\Delta\boldsymbol{x}_k$，返回2

优缺点：

- 非线性优化中，相当多算法都可以归结为高斯牛顿法的变种。
- 可能出现$\boldsymbol{J}_T\boldsymbol{J}$为奇异矩阵或者病态矩阵的情况，此时增量的稳定性较差，导致算法不收敛。
- 如果求出的步长$\Delta\boldsymbol{x}$太大，导致局部近似不准确，甚至可能无法保证迭代收敛。

2.2.3 列文伯格-马垮尔特法(L-M)

改进点：

G-N法中采用近似二阶泰勒展开只能在展开点附近有较好的近似效果，因此想到给$\Delta\boldsymbol{x}$添加一个信赖区域，使用实际函数和近似模型之间的差异来确定信赖区域的范围。若ρ接近1，则近似是好的；若太小，则减小近似范围；反之增大近似范围。

$$\rho = \frac{f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})}{\boldsymbol{J}\Delta\boldsymbol{x}}$$

推导：

- 给定初值$\boldsymbol{x}_0$，以及初始化半径μ
- 对于第k次迭代，求解如下问题：
    - μ：信赖区域的半径
    - $\boldsymbol{D}$：取为单位矩阵$\boldsymbol{I}$，相当于把$\Delta\boldsymbol{x}$约束在一个球中；取为非负数对角阵（常用$\boldsymbol{J}_T\boldsymbol{J}$的对角元素平方根），使得在梯度小的维度上约束范围更大。

$$\min_{\Delta\boldsymbol{x}_k}\frac{1}{2}||f(\boldsymbol{x}_k)+\boldsymbol{J}(\boldsymbol{x}_k)\Delta\boldsymbol{x}_k||_2,\ \ s.t.||\boldsymbol{D}\Delta\boldsymbol{x}_k||_2\le\mu$$

    - 利用拉格朗日乘子λ将上式转换为无约束的优化问题：

$$\min_{\Delta\boldsymbol{x}_k}\frac{1}{2}||f(\boldsymbol{x}_k)+\boldsymbol{J}(\boldsymbol{x}_k)\Delta\boldsymbol{x}_k||_2+\frac{\lambda}{2}||\boldsymbol{D}\Delta\boldsymbol{x}||_2$$

    - 类似G-N的做法，展开后计算：

$$(\boldsymbol{H}+\lambda\boldsymbol{D}_T\boldsymbol{D})\Delta\boldsymbol{x}=\boldsymbol{g}$$

    - 相比G-N，多了一项$\lambda\boldsymbol{D}_T\boldsymbol{D}$，若$\boldsymbol{D}=\boldsymbol{I}$，则转为求解：

$$(\boldsymbol{H}+\lambda\boldsymbol{I})\Delta\boldsymbol{x}=\boldsymbol{g}$$

- 计算ρ。
- 若$\rho>\frac{3}{4}$，则$\mu=2\mu$。
- 若$\rho<\frac{1}{4}$，则$\mu=0.5\mu$。
- 如果ρ大于某阈值，认为近似可行。令$\boldsymbol{x}_{k+1}=\boldsymbol{x}_k+\Delta\boldsymbol{x}_k$。
- 判断算法是否收敛。如不收敛则返回2，否则结束。

优缺点：

- 当参数λ较小时，二次近似模型在该范围较好，L-M法更接近G-N法。
- 当λ较大时，附近的二次近似不好，L-M更接近一阶梯度下降法。

第3章 Ceres库｜求解最小二乘

3.1 Ceres Solver库

官网:

http://ceres-solver.org/

安装：

```
sudo apt-get install liblapack-dev libsuitesparse-dev libcxsparse3.1.4 libgflags-dev libgo
git clone https://github.com/ceres-solver/ceres-solver.git
cd ceres-solver/
mkdir build
cd build
cmake ..
make
make install
```

Ceres求解最小二乘问题：

Ceres库面向通用的最小二乘问题的求解，用户需要做的就是定义优化问题，然后设置一些选项，输入进Ceres求解即可。步骤一般包含：定义代价函数计算模型；构建最小二乘问题；配置求解器。

- 核函数：ρ
- 代价函数：f_i
- 最简情况：取$l_j = -\infty$，$u_j = \infty$，ρ为恒等函数，得到无约束的最小二乘问题。

$$\min_{x} \quad \frac{1}{2}\sum \rho_i\left[\left\|f_i(x_{i_1},\cdots,x_{i_n})\right\|\right]$$
$$s.t. \quad l_j \le x_j \le u_j$$

3.2 拟合曲线

完整代码：https://gitee.com/deng-ke/slam_notes/tree/master/ceres_curve_fitting

- 目标:使用Ceres拟合如下曲线

$$y = exp(ax_2 + bx + c) + \omega$$

- 输入：N对x、y；高斯噪声ω。
- 输出：曲线参数a、b、c。
- 方法:求如下最小二乘问题以估计曲线参数。（相当于机器学习中最小化代价函数）

$$\min_{a,b,c} \frac{1}{2}\sum_{i=1}^{N} ||y_i - exp(ax_{i2} + bx_i + c)||_2$$

3.2.1 定义代价函数计算模型

以仿函数形式，重载()，定义代价函数。

```
/**************************************************************************
 *定义代价函数的计算模型
 **************************************************************************/
struct CURVE_FITTING_COST
{
    CURVE_FITTING_COST ( double x, double y ) : _x ( x ), _y ( y ) {}

    /*残差的计算（重载()，仿函数）*/
    template <typename T>
    bool operator() (
        const T* const abc,  // 模型参数，有3维
        T* residual ) const  // 残差
    {
        residual[0] = T ( _y ) - ceres::exp ( abc[0]*T ( _x ) *T ( _x ) + abc[1]*T ( _x )
        return true;
    }

    //x、y数据
    const double _x, _y;
};
```

3.2.2 生成数据

根据真实的模型生成x、y真值，再加入随机的高斯噪声。

```
/****************************************************************************
 *生成数据
 ****************************************************************************/
    double a=1.0, b=2.0, c=1.0;          // 真实参数值
    int N=100;                           // 数据点
    double w_sigma=1.0;                  // 噪声Sigma值
    cv::RNG rng;                         // OpenCV随机数产生器
    double abc[3] = {0,0,0};             // abc参数的估计值
    vector<double> x_data, y_data;       // 数据

    /*根据真实的模型生成x、y真值，再加入随机的高斯噪声*/
    cout<<"generating data: "<<endl;
    for ( int i=0; i<N; i++ )
    {
        double x = i/100.0;
        x_data.push_back ( x );
        y_data.push_back (
            exp ( a*x*x + b*x + c ) + rng.gaussian ( w_sigma )
        );
        cout<<x_data[i]<<" "<<y_data[i]<<endl;
    }
```

3.2.3 构建最小二乘问题

导入前面定义的代价函数模型。

```
/****************************************************************************
 *构建最小二乘问题
 ****************************************************************************/
    ceres::Problem problem;
    for ( int i=0; i<N; i++ )
    {
        problem.AddResidualBlock (      // 向问题中添加误差项
        // 使用自动求导，模板参数：误差类型，输出维度，输入维度。（维数要与前面struct中一致）
            new ceres::AutoDiffCostFunction<CURVE_FITTING_COST, 1, 3> (
                new CURVE_FITTING_COST ( x_data[i], y_data[i] )
            ),
            nullptr,            // 核函数，这里不使用，为空
            abc                 // 待估计参数
        );
    }
```

3.2.4 配置求解器

```
/*****************************************************************************
 *配置求解器
 *****************************************************************************/
    ceres::Solver::Options options;     // 这里有很多配置项可以填
    options.linear_solver_type = ceres::DENSE_QR;  // 增量方程如何求解
    options.minimizer_progress_to_stdout = true;   // 输出到cout

    ceres::Solver::Summary summary;                // 优化信息
    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    ceres::Solve ( options, &problem, &summary );  // 开始优化
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(
    cout<<"solve time cost = "<<time_used.count()<<" seconds. "<<endl;

    // 输出结果
    cout<<summary.BriefReport() <<endl;
    cout<<"estimated a,b,c = ";
    for ( auto a:abc ) cout<<a<<" ";
    cout<<endl;
```

3.2.5 结果

```
iter      cost      cost_change  |gradient|   |step|    tr_ratio  tr_radius  ls_iter  iter_time  total_time
   0  1.824887e+04    0.00e+00    1.38e+03   0.00e+00   0.00e+00  1.00e+04        0    3.98e-05    1.48e-04
   1  2.748700e+39   -2.75e+39    0.00e+00   7.67e+01  -1.52e+35  5.00e+03        1    5.29e-05    3.17e-04
   2  2.429783e+39   -2.43e+39    0.00e+00   7.62e+01  -1.35e+35  1.25e+03        1    2.41e-05    3.96e-04
   3  1.213227e+39   -1.21e+39    0.00e+00   7.30e+01  -6.73e+34  1.56e+02        1    2.31e-05    5.10e-04
   4  1.852387e+37   -1.85e+37    0.00e+00   5.56e+01  -1.03e+33  9.77e+00        1    2.22e-05    5.86e-04
   5  6.714689e+31   -6.71e+31    0.00e+00   2.96e+01  -3.85e+27  3.05e-01        1    2.41e-05    6.65e-04
   6  9.500531e+12   -9.50e+12    0.00e+00   9.50e+00  -8.39e+08  4.77e-03        1    2.10e-05    7.38e-04
   7  1.776982e+04    4.79e+02    1.83e+03   2.58e-01   1.18e+00  1.43e-02        1    4.79e-05    8.37e-04
   8  1.599969e+04    1.77e+03    3.45e+03   5.53e-01   1.46e+00  4.29e-02        1    4.41e-05    9.33e-04
   9  1.060557e+04    5.39e+03    7.62e+03   7.33e-01   1.68e+00  1.29e-01        1    4.29e-05    1.03e-03
  10  3.669783e+03    6.94e+03    9.60e+03   5.25e-01   1.39e+00  3.86e-01        1    4.20e-05    1.12e-03
  11  5.397541e+02    3.13e+03    5.00e+03   2.66e-01   1.12e+00  1.16e+00        1    4.20e-05    1.21e-03
  12  1.484444e+02    3.91e+02    1.22e+03   8.46e-02   1.02e+00  3.48e+00        1    4.20e-05    1.30e-03
  13  1.216815e+02    2.68e+01    3.76e+02   4.17e-02   1.01e+00  1.04e+01        1    4.20e-05    1.40e-03
  14  9.290109e+01    2.88e+01    2.42e+02   9.10e-02   1.01e+00  3.13e+01        1    4.79e-05    1.52e-03
  15  6.674330e+01    2.62e+01    1.09e+02   1.33e-01   1.00e+00  9.39e+01        1    5.51e-05    1.65e-03
  16  5.936574e+01    7.38e+00    2.14e+01   1.08e-01   9.94e-01  2.82e+02        1    4.41e-05    1.75e-03
  17  5.653118e+01    2.83e+00    1.36e+01   1.57e-01   9.98e-01  8.45e+02        1    4.29e-05    1.84e-03
  18  5.310764e+01    3.42e+00    8.50e+00   2.81e-01   9.89e-01  2.53e+03        1    4.20e-05    1.94e-03
  19  5.125939e+01    1.85e+00    2.84e+00   2.98e-01   9.90e-01  7.60e+03        1    4.29e-05    2.03e-03
  20  5.097693e+01    2.82e-01    4.34e-01   1.48e-01   9.95e-01  2.28e+04        1    4.20e-05    2.12e-03
  21  5.096854e+01    8.39e-03    3.24e-02   2.87e-02   9.96e-01  6.84e+04        1    4.20e-05    2.22e-03
solve time cost = 0.00230917 seconds.
Ceres Solver Report: Iterations: 22, Initial cost: 1.824887e+04, Final cost: 5.096854e+01, Termination: CONVERGENCE
estimated a,b,c = 0.891943 2.17039 0.944142
```

第4章 优化 | EKF滤波器

4.1 卡尔曼滤波的引出

$$P(\boldsymbol{x}_k|\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{1:k-1})=\int P(\boldsymbol{x}_k|\boldsymbol{x}_{k-1},\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{1:k-1})P(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{1:k-1})d\boldsymbol{x}_{k-1}$$

马尔可夫性：k时刻状态只与$k-1$时刻状态有关。将如下公式简化：

- 右侧第一部分：对应k时刻的运动方程。

$$P(\boldsymbol{x}_k|\boldsymbol{x}_{k-1},\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{1:k-1})=P(\boldsymbol{x}_k|\boldsymbol{x}_{k-1},\boldsymbol{u}_k)$$

- 右侧第二部分：k-1时刻的状态分布。

$$P(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{1:k-1})=P(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0,\boldsymbol{u}_{1:k-1},\boldsymbol{z}_{1:k-1})$$

因此，KF、EKF：把$k-1$时刻的状态分布推导至k时刻。（程序运行期间，只要维护一个状态量，对它进行不断地迭代和更新即可）

4.2 线性系统和KF

线性系统：运动、观测方程可由线性方程表示：

$$\begin{cases}\boldsymbol{x}_k=\boldsymbol{A}_k\boldsymbol{x}_{k-1}+\boldsymbol{u}_k+\boldsymbol{\omega}_k,\quad \boldsymbol{\omega}\sim N(\boldsymbol{0},\boldsymbol{R})\\ \boldsymbol{z}_k=\boldsymbol{C}_k\boldsymbol{x}_k+\boldsymbol{v}_k,\quad \boldsymbol{v}_k\sim N(\boldsymbol{0},\boldsymbol{Q})\end{cases}$$

KF目标：假设已知$k-1$时刻的后验状态估计：$\hat{\boldsymbol{x}}^{k-1}$和它的协方差$\hat{\boldsymbol{P}}^{k-1}$。根据$k$时刻的输入和观测数据，确定的后验分布$\hat{\boldsymbol{x}}^k$。（记：$\hat{\boldsymbol{x}}^k$后验，$\overline{\boldsymbol{x}}$先验，$\hat{\boldsymbol{P}}^k$协方差）

4.2.1 推导

预测：

从运动方程出发，根据高斯分布性质：

$$P(\boldsymbol{x}_k|\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{1:k-1})=N(\boldsymbol{A}_{k}\hat{\boldsymbol{x}}^{k-1}+\boldsymbol{u}_k,\boldsymbol{A}_{k}\hat{\boldsymbol{P}}^{k-1}\boldsymbol{A}_{k}^{T}+\boldsymbol{R})$$

从上一个时刻的状态$\hat{\boldsymbol{x}}^{k-1}$，根据输入信息（有噪声），推断当前时刻的状态分布$\overline{\boldsymbol{x}}^{k}$。得到预测方程：

$$\overline{\boldsymbol{x}}=\boldsymbol{A}_{k}\hat{\boldsymbol{x}}^{k-1}+\boldsymbol{u}_k,\quad \overline{\boldsymbol{P}}=\boldsymbol{A}_{k}\hat{\boldsymbol{P}}^{k-1}\boldsymbol{A}_{k}^{T}+\boldsymbol{R}$$

更新：

从观测方程出发，可计算在某个状态下，应该产生怎样的观测数据。

$$P(\boldsymbol{z}_k|\boldsymbol{x}_k)=N(\boldsymbol{C}_k\boldsymbol{x}_k,\boldsymbol{Q})$$

4.2.2 总结

4.3 非线性系统和EKF

SLAM中运动、观测方程通常是非线性的，非线性系统：